

BIG DATA Y MACHINE LEARNING

Universidad de
los AndesFacultad
de EconomíaEducación
Continúa
INSTRUMENTOS ACADÉMICOS

Estimating Generalization (or Out-of-Sample) Error with `caret`

AUTHORS

Julián David Rojas Aguilar

Ignacio Sarmiento-Barbieri

AFFILIATION

Universidad de los Andes

Universidad de los Andes

PUBLISHED

2026-08-28

A model may reproduce its training data remarkably well and still make poor predictions for new observations. Therefore, a predictive model is useful when it performs “well” on observations that were not used to estimate it. This problem becomes especially important as model complexity increases: additional variables, polynomials, and interactions can capture meaningful patterns, but they can also adapt to noise and other sample-specific features.

Because of this, we should not select a predictive model by its in-sample fit. We instead want to estimate its **generalization error**—i.e., the prediction error we expect to make on new observations drawn from the same population—. Given that these new observations are not available when we estimate the model, we approximate this error by holding aside part of the observed sample for evaluation. Resampling methods formalize this idea and allow us to compare models according to their out-of-sample performance¹.

How Can We Estimate Generalization Error?

In this lecture, we study three methods: the validation set approach, K -fold cross-validation, and leave-one-out cross-validation (LOOCV). We use the [caret](#) package to implement this workflow. Its name stands for **C**lassification **A**nd **R**egression **T**raining, and it provides functions for data partitioning, resampling, model estimation, prediction, and performance assessment. The package automates much of the computation, but the substantive task remains ours: we must choose a suitable resampling design and interpret what the resulting estimates imply about model performance.

Learning Objectives

By the end of this lecture, you should be able to:

1. Distinguish between in-sample and out-of-sample prediction error and explain why training error could be optimistic.
2. Explain how model complexity relates to underfitting, an appropriate fit, and overfitting.
3. Implement the validation set approach, K -fold cross-validation, and LOOCV with `caret`.
4. Calculate and interpret the root mean squared error (RMSE).
5. Compare nested models using their estimated generalization errors and identify evidence of overfitting.
6. Explain the principal trade-offs among the three resampling methods, including sensitivity to random partitions and computational cost.

Acknowledgments: Much of the material presented here was contributed by previous teaching assistants, especially [Eduard Martínez](#), [Lucas Gómez](#), Andrés Rengifo, Valentina Laverde, Juan Segura, and Gustavo Castillo. We are truly grateful for their help.

What Is the Generalization Error?

Remember that we distinguish between two types of prediction error:

1. **In-sample error.** The prediction error calculated using the observations employed to estimate the model.
2. **Out-of-sample error.** The prediction error the model makes on new observations drawn from the same population.

As model complexity increases, the model has more freedom to adapt to the data it has observed. Its in-sample error therefore tends to decrease. The **out-of-sample error**, however, behaves differently. At first, it also decreases because some additional flexibility helps the model capture the relationship between X and Y . Nevertheless, beyond a certain point, further complexity causes the out-of-sample error to increase again.

We can consequently distinguish three regions:

- **Underfitting.** The model is too simple to capture the relationship in the data. Both its in-sample and out-of-sample errors are high.
- **Appropriate fit.** The model is flexible enough to capture the signal without adapting to sample-specific features. Its out-of-sample error reaches its minimum.
- **Overfitting.** The model continues to improve its in-sample fit, but its predictions for new observations become worse.

Our goal is not to find the model that best reproduces the data we have already observed. Instead, we seek a level of complexity that **captures systematic patterns** without **learning idiosyncratic features of the sample**.

Empirical Example: Hours Worked

Suppose we want to study labor supply at the intensive margin: among people who work, how do observed characteristics relate to their total hours worked? We are particularly interested in gender differences and whether the association between household composition and hours differs between men and women.

A general model could express the hours worked by individual i in household f as a function of their labor income², individual demographic characteristics (such as age, gender, educational attainment, and whether they are the household head), and family characteristics (such as the number of minors in the household). However, we do not know the functional form of $\mathbb{E}[Hours_{if} \mid X_{if}]$. We can therefore compare alternative specifications using their estimated out-of-sample errors and retain the specification with the lowest value.

Initial Configuration

We use the `pacman` package to simplify package management.

▼ Show/hide code

```
1 # Install pacman if necessary, then load the packages used in this session.
2 if (!require(pacman)) install.packages("pacman")
3 library(pacman)
4
5 p_load(rio,      # Import data from a URL.
6         tidyverse, # Manipulate data and create figures.
7         caret,    # For predictive model assessment.
8         conflicted) # Resolve conflicts between functions with the same name.
9
10 conflict_prefer(name = "select", winner = "dplyr")
11 conflict_prefer(name = "filter", winner = "dplyr")
12
13 options(scipen = 999)
```

Importing Data

We import a subset of the GEIH data that will be used in the first problem set.

▼ Show/hide code

```
1 db_geih <- rio::import("https://raw.githubusercontent.com/ignaciomsarmiento/datasets/main/GE
2   as_tibble()

```

The sample contains people interviewed in Bogotá. The combination of building, household, and person identifiers uniquely identifies each respondent. The [data dictionary](#) documents the survey

variables.

Cleaning the Dataset

Our unit of analysis is an employed adult with positive reported hours worked. Before imposing that individual-level restriction, we construct household variables using all observed household members.

▼ Show/hide code

```
1  # The names of this vector are the standardized names used below. Its values
2  # are the corresponding names in the original data.
3  text_vars <- c('id_building' = 'directorio', 'id_household' = 'secuencia_p',
4                'id_person' = 'orden', 'num_age' = 'age', 'bin_male' = 'sex',
5                'bin_employed' = 'ocu', 'bin_formal' = 'formal',
6                'cat_educ' = 'maxEducLevel', 'num_hoursWorked' = 'totalHoursWorked',
7                'cat_job' = 'oficio', 'cat_relationship' = 'p6050',
8                'num_labIncome' = 'y_total_m')
9
10 db_geih <- db_geih |>
11   select(all_of(text_vars))
```

Number of Minors per Household

We count household members younger than 18 before restricting the sample to employed adults³.

▼ Show/hide code

```
1  db_geih <- db_geih |>
2    mutate(bin_minor = ifelse(test = num_age < 18, yes = 1, no = 0)) |>
3    group_by(id_building, id_household) |>
4    mutate(num_minors = sum(bin_minor, na.rm = TRUE)) |>
5    ungroup() |>
6    select(-bin_minor)
```

Household Head and Sample Restriction

We identify the household head and then retain employed respondents aged 18 or older with positive reported hours. We postpone the complete-case restriction until after recoding education and income. In applied work, however, you should document the causes and patterns of missingness rather than dropping observations by default.

▼ Show/hide code

```

1 db_geih <- db_geih |>
2   mutate(bin_head = ifelse(test = cat_relationship == 1, yes = 1, no = 0)) |>
3   filter(bin_employed == 1, num_age >= 18, num_hoursWorked > 0)

```

Treatment for Categorical Variables: the Case of Educational Level

`cat_educ` has several missing values. For this illustration, we assign respondents who do not report an education level to the `None` category. This is a strong assumption. In your own empirical applications, you should investigate the causes of missingness and impute missing values only under a defensible missing-data strategy.

▼ Show/hide code

```

1 db_geih <- db_geih |>
2   mutate(cat_educ = case_when(is.na(cat_educ) ~ 1, # If education is missing, impute 1.
3                               cat_educ == 9 ~ 1,   # If the respondent does not know, impute
4                               TRUE ~ cat_educ),    # Otherwise, we leave the original value.
5   cat_educ = factor(x = cat_educ,
6                     levels = 1:7,
7                     labels = c("None", "Preschool", "Primary Incomplete",
8                                "Primary Complete", "Secondary Incomplete",
9                                "Secondary Complete", "Tertiary")),
10   # We define `None` as the reference level. Otherwise, R would select the
11   # reference level alphabetically.
12   cat_educ = fct_relevel(cat_educ, "None"))

```

Rescaling Income

We express `num_labIncome` in millions of Colombian pesos to improve numerical stability⁴. Finally, we impose the complete-case restriction.

▼ Show/hide code

```

1 db_geih <- db_geih |>
2   mutate(num_labIncome = num_labIncome / 1000000) |>
3   filter(num_labIncome > 0) |>
4   drop_na()

```

Model Assessment via Resampling Methods with `caret`

Let $Hours_{if}$ denote the hours worked by individual i in household f , E_i their educational level, $Male_i$ an indicator for being male, $Minors_f$ the number of minors in their household, $Income_i$ their total labor income, and Age_i their age. We use the `caret` package to compare the prediction errors of seven nested linear models. Each model adds one variable (or a substantively motivated group of terms) to the preceding specification. We measure predictive performance using the root mean squared error (RMSE). Because RMSE is expressed in the same units as the outcome—weekly hours—we can interpret its magnitude directly.

Note

We define the seven formulas once and reuse the same objects in every resampling method. The `:` operator adds only an interaction, whereas `*` adds both the main effects and the interaction.

Model 0: benchmark (unconditional mean). The intercept-only model predicts the same value for every observation: the unconditional mean. Model 0 therefore provides a useful benchmark: a more complex model should predict better than simply assigning the mean to everyone.

$$Hours_{if} = \alpha + \varepsilon_{if}$$

Warning

The `caret` package requires the design matrix to contain at least one named column. For Model 0, we therefore create `num_constant`, a column equal to one, and remove R's automatic intercept with `0 + num_constant`. Its sole estimated coefficient is still the sample mean, but this implementation is necessary because of the package constraint.

▼ Show/hide code

```
1 db_geih <- db_geih |> mutate(num_constant = 1)
2 formula_m0 <- num_hoursWorked ~ 0 + num_constant
```

Model 1: log income. Labor income is likely to contain substantial predictive information about hours worked. The logarithmic transformation comes from the solution to the labor supply in a microfounded model.

$$Hours_{if} = \alpha + \beta_1 \log(Income_i) + \varepsilon_{if}$$

▼ Show/hide code

```
1 formula_m1 <- num_hoursWorked ~ log(num_labIncome)
```

Starting with Model 2, we add variables that we expect to have predictive power for hours worked.

Model 2: educational-level fixed effects. The term γ_{E_i} represents educational-level fixed effects.

$$Hours_{if} = \alpha + \beta_1 \log(Income_i) + \gamma_{E_i} + \varepsilon_{if}$$

In the R formula, `None` is the omitted or base category. Each reported education coefficient therefore measures a difference relative to `None`, holding income constant.

▼ Show/hide code

```
1 formula_m2 <- num_hoursWorked ~ log(num_labIncome) + cat_educ
```

Model 3: gender.

$$Hours_{if} = \alpha + \beta_1 \log(Income_i) + \gamma_{E_i} + \beta_2 Male_i + \varepsilon_{if}$$

▼ Show/hide code

```
1 formula_m3 <- num_hoursWorked ~ log(num_labIncome) + cat_educ + bin_male
```

Model 4: number of minors.

$$Hours_{if} = M3 + \beta_3 Minors_f + \varepsilon_{if}$$

▼ Show/hide code

```
1 formula_m4 <- num_hoursWorked ~ log(num_labIncome) + cat_educ + bin_male +  
2   num_minors
```

Model 5: gender-specific association between minors and hours worked. The interaction allows the association between household composition and hours worked to differ by gender.

$$Hours_{if} = M4 + \beta_4 (Male_i \times Minors_f) + \varepsilon_{if}$$

▼ Show/hide code

```
1 # Note that an alternative way to define this formula is:  
2 # formula_m5 <- num_hoursWorked ~ log(num_labIncome) + cat_educ +  
3 #   bin_male * num_minors  
4 formula_m5 <- num_hoursWorked ~ log(num_labIncome) + cat_educ + bin_male +  
5   num_minors + bin_male:num_minors
```

Model 6: deliberately flexible life-cycle profiles. The most complex model adds a fifth-degree age profile and allows that profile to differ by gender, education, and the number of minors:

$$\begin{aligned}
Hours_{if} = & M5 + \sum_{r=1}^5 \theta_r Age_i^r + \sum_{r=1}^5 \theta_r^M (Male_i \times Age_i^r) \\
& + \sum_{e \neq e_0} \sum_{r=1}^5 \theta_{er}^E [\mathbb{1}(E_i = e) \times Age_i^r] \\
& + \sum_{r=1}^5 \theta_r^C (Minors_f \times Age_i^r) + \varepsilon_{if}
\end{aligned}$$

Hours worked may follow a nonlinear life-cycle pattern, and that pattern may differ by gender, educational attainment, and caregiving intensity. We deliberately use a fifth-degree polynomial to create a highly flexible specification whose training fit can improve even when its predictions for unseen observations deteriorate. Such a high-degree model would require stronger substantive and empirical justification in an applied analysis. Here, it serves a pedagogical purpose.

▼ Show/hide code

```

1 formula_m6 <- num_hoursWorked ~ log(num_labIncome) + cat_educ + bin_male +
2   num_minors + bin_male:num_minors +
3   poly(num_age, 5) + poly(num_age, 5):bin_male + poly(num_age, 5):cat_educ +
4   poly(num_age, 5):num_minors

```

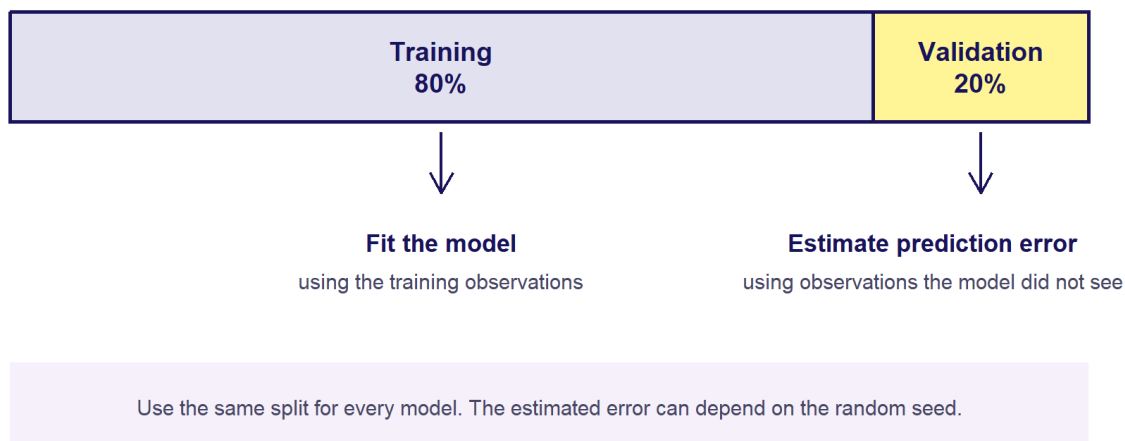
Validation Set Approach

We first split the dataset into training and validation sets. We fit each model on the training set and evaluate its predictions on the same held-out validation set. This method is quick and simple, but its estimate of the generalization error can vary considerably with the initial random split. We revisit this sensitivity at the end of the lecture.

Validation-set approach: fit the model on 80% of the sample and evaluate it on the remaining 20%.

Validation-set approach

One random split of the available sample



We use `caret::createDataPartition()` to assign 80% of the observations to the training set⁵. Supplying the outcome helps preserve its empirical distribution across the two samples.

▼ Show/hide code

```
1 set.seed(11)
2
3 in_train <- createDataPartition(y = db_geih$num_hoursWorked, p = 0.80, list = FALSE)
```

We use this one partition for all seven models. Otherwise, differences in RMSE could reflect different validation observations rather than different model specifications.

▼ Show/hide code

```
1 training <- db_geih[in_train, ]
2 testing <- db_geih[-in_train, ]
```

Train and Evaluate the Models

Model 0

▼ Show/hide code

```
1 model_m0_training <- lm(formula = formula_m0, data = training)
2
3 predictions_m0_training <- predict(object = model_m0_training)
4
```

```

5  score_m0_training <- RMSE(pred = predictions_m0_training,
6                             obs = training$num_hoursWorked)
7
8  predictions_m0_validation <- predict(object = model_m0_training,
9                                       newdata = testing)
10
11 score_m0_validation <- RMSE(pred = predictions_m0_validation,
12                              obs = testing$num_hoursWorked)
13
14 print(c('Training RMSE' = score_m0_training,
15         'Validation RMSE' = score_m0_validation))

```

Training RMSE	Validation RMSE
14.09585	13.96785

Model 0 predicts the training-sample mean for every validation observation. Its RMSE is the benchmark against which we evaluate the additional predictors. In RMSE terms, its predictions differ from the observed hours worked by approximately 13.97 hours.

Model 1

▼ Show/hide code

```

1  model_m1_training <- lm(formula = formula_m1, data = training)
2
3  predictions_m1_training <- predict(object = model_m1_training)
4
5  score_m1_training <- RMSE(pred = predictions_m1_training,
6                             obs = training$num_hoursWorked)
7
8  predictions_m1_validation <- predict(object = model_m1_training,
9                                       newdata = testing)
10
11 score_m1_validation <- RMSE(pred = predictions_m1_validation,
12                              obs = testing$num_hoursWorked)
13
14 print(c('Training RMSE' = score_m1_training,
15         'Validation RMSE' = score_m1_validation))

```

Training RMSE	Validation RMSE
13.67211	13.37092

Both the in-sample and out-of-sample RMSE values decrease with this specification. The model is more complex, but the additional predictor still captures a pattern that generalizes beyond the training sample—that is, part of the relationship between X and Y .

Model 2

▼ Show/hide code

```

1  model_m2_training <- lm(formula = formula_m2, data = training)
2
3  predictions_m2_training <- predict(object = model_m2_training)
4
5  score_m2_training <- RMSE(pred = predictions_m2_training,
6                             obs = training$num_hoursWorked)
7
8  predictions_m2_validation <- predict(object = model_m2_training,
9                                       newdata = testing)
10
11 score_m2_validation <- RMSE(pred = predictions_m2_validation,
12                              obs = testing$num_hoursWorked)
13
14 print(c('Training RMSE' = score_m2_training,
15          'Validation RMSE' = score_m2_validation))

```

Training RMSE	Validation RMSE
13.11771	12.91019

Model 3

▼ Show/hide code

```

1  model_m3_training <- lm(formula = formula_m3, data = training)
2
3  predictions_m3_training <- predict(object = model_m3_training)
4
5  score_m3_training <- RMSE(pred = predictions_m3_training,
6                             obs = training$num_hoursWorked)
7
8  predictions_m3_validation <- predict(object = model_m3_training,
9                                       newdata = testing)
10
11 score_m3_validation <- RMSE(pred = predictions_m3_validation,
12                              obs = testing$num_hoursWorked)
13
14 print(c('Training RMSE' = score_m3_training,
15          'Validation RMSE' = score_m3_validation))

```

Training RMSE	Validation RMSE
13.05506	12.77091

Model 4

▼ Show/hide code

```

1  model_m4_training <- lm(formula = formula_m4, data = training)
2
3  predictions_m4_training <- predict(object = model_m4_training)
4

```

```

5  score_m4_training <- RMSE(pred = predictions_m4_training,
6                             obs = training$num_hoursWorked)
7
8  predictions_m4_validation <- predict(object = model_m4_training,
9                                       newdata = testing)
10
11 score_m4_validation <- RMSE(pred = predictions_m4_validation,
12                             obs = testing$num_hoursWorked)
13
14 print(c('Training RMSE' = score_m4_training,
15         'Validation RMSE' = score_m4_validation))

```

Training RMSE	Validation RMSE
13.03778	12.75316

Although both RMSE measures decrease, the improvements are small. This pattern suggests that we are approaching the flat portion of the error curve.

Model 5

▼ Show/hide code

```

1  model_m5_training <- lm(formula = formula_m5, data = training)
2
3  predictions_m5_training <- predict(object = model_m5_training)
4
5  score_m5_training <- RMSE(pred = predictions_m5_training,
6                             obs = training$num_hoursWorked)
7
8  predictions_m5_validation <- predict(object = model_m5_training,
9                                       newdata = testing)
10
11 score_m5_validation <- RMSE(pred = predictions_m5_validation,
12                             obs = testing$num_hoursWorked)
13
14 print(c('Training RMSE' = score_m5_training,
15         'Validation RMSE' = score_m5_validation))

```

Training RMSE	Validation RMSE
13.03567	12.72716

Again, we observe only a small improvement. Have we already reached the “sweet spot”?

Model 6

▼ Show/hide code

```

1  model_m6_training <- lm(formula = formula_m6, data = training)
2
3  predictions_m6_training <- predict(object = model_m6_training)

```

```

4
5  score_m6_training <- RMSE(pred = predictions_m6_training,
6                             obs = training$num_hoursWorked)
7
8  predictions_m6_validation <- predict(object = model_m6_training,
9                                       newdata = testing)
10
11 score_m6_validation <- RMSE(pred = predictions_m6_validation,
12                             obs = testing$num_hoursWorked)
13
14 print(c('Training RMSE' = score_m6_training,
15         'Validation RMSE' = score_m6_validation))

```

Training RMSE	Validation RMSE
12.53025	13.17217

Finally, although the in-sample fit continues to improve, our estimate of the generalization error increases. This divergence indicates that Model 6 is overfitting.

Interpretation of the RMSE

The training RMSE decreases from 14.1 hours in Model 0 to 12.53 hours in Model 6. This pattern is expected because the models are nested and estimated by OLS on the same training sample: additional terms cannot increase the training sum of squared residuals.

The validation RMSE initially follows the same direction. It falls from Model 0 to Model 1 and then continues decreasing as Models 2–5 add education, gender, the number of minors, and the gender–minors interaction. Model 5 reaches the lowest validation RMSE. In Model 6, however, the validation RMSE rises. This pattern indicates that the fifth-degree polynomials and their interactions improve the model's ability to reproduce the training sample but weaken its predictions for unseen observations.

K-Fold Cross-Validation

To perform K -fold cross-validation, we randomly divide the dataset into K subsets, or folds, of approximately the same size. We train and evaluate each model K times. In every iteration, a different fold acts as the validation set and the remaining $K - 1$ folds form the training set.

Five-fold cross-validation: each fold serves as the validation set once.

Five-fold cross-validation

Rotate the validation fold while keeping the partition fixed

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Run 1	Validate	Train	Train	Train	Train
Run 2	Train	Validate	Train	Train	Train
Run 3	Train	Train	Validate	Train	Train
Run 4	Train	Train	Train	Validate	Train
Run 5	Train	Train	Train	Train	Validate

Each fold is used once for validation. The reported RMSE averages the five held-out errors.

We create the five folds once and pass the same training indices to all seven models. This pairing ensures that every model is assessed on identical observations in each fold.

▼ Show/hide code

```

1  set.seed(11)
2
3  # The object `index_kfold` contains a list of five elements: the five folds.
4  # Each vector (or element) of the list contains the indices of training
5  # observations in each fold.
6  index_kfold <- createFolds(y = db_geih$num_hoursWorked, k = 5, returnTrain = TRUE)
7
8  ctrl_kfold <- trainControl(
9    method = "cv",
10   number = 5,
11   index = index_kfold,
12   savePredictions = "final"
13 )

```

Train and Evaluate the Models

We supply `ctrl_kfold` through the `trControl` argument of `train()`. The function then estimates out-of-sample error through cross-validation. For each model, the cross-validation RMSE is the mean of

the five fold-specific held-out RMSE values. In this exercise, we focus on the estimated out-of-sample error.

Model 0

▼ Show/hide code

```
1  # The object `model_m0_kfold` is a list that contains several results.
2  # The element of interest is `resample`, which contains the performance
3  # metrics.
4  model_m0_kfold <- train(
5    formula_m0,
6    data = db_geih,
7    method = "lm",
8    metric = "RMSE",
9    trControl = ctrl_kfold
10 )
11
12 score_m0_kfold <- mean(model_m0_kfold$resample$RMSE)
13
14 print(c('Cross-validation RMSE' = score_m0_kfold))
```

```
Cross-validation RMSE
14.05813
```

Model 1

▼ Show/hide code

```
1  model_m1_kfold <- train(
2    formula_m1,
3    data = db_geih,
4    method = "lm",
5    metric = "RMSE",
6    trControl = ctrl_kfold
7  )
8
9  score_m1_kfold <- mean(model_m1_kfold$resample$RMSE)
10
11 print(c('Cross-validation RMSE' = score_m1_kfold))
```

```
Cross-validation RMSE
13.61352
```

Model 2

▼ Show/hide code

```
1  model_m2_kfold <- train(
2    formula_m2,
```

```
3 data = db_geih,  
4 method = "lm",  
5 metric = "RMSE",  
6 trControl = ctrl_kfold  
7 )  
8  
9 score_m2_kfold <- mean(model_m2_kfold$resample$RMSE)  
10  
11 print(c('Cross-validation RMSE' = score_m2_kfold))
```

Cross-validation RMSE
13.18903

Model 3

▼ Show/hide code

```
1 model_m3_kfold <- train(  
2 formula_m3,  
3 data = db_geih,  
4 method = "lm",  
5 metric = "RMSE",  
6 trControl = ctrl_kfold  
7 )  
8  
9 score_m3_kfold <- mean(model_m3_kfold$resample$RMSE)  
10  
11 print(c('Cross-validation RMSE' = score_m3_kfold))
```

Cross-validation RMSE
13.125

Model 4

▼ Show/hide code

```
1 model_m4_kfold <- train(  
2 formula_m4,  
3 data = db_geih,  
4 method = "lm",  
5 metric = "RMSE",  
6 trControl = ctrl_kfold  
7 )  
8  
9 score_m4_kfold <- mean(model_m4_kfold$resample$RMSE)  
10  
11 print(c('Cross-validation RMSE' = score_m4_kfold))
```

Cross-validation RMSE
13.13097

Model 5

▼ Show/hide code

```
1  model_m5_kfold <- train(
2    formula_m5,
3    data = db_geih,
4    method = "lm",
5    metric = "RMSE",
6    trControl = ctrl_kfold
7  )
8
9  score_m5_kfold <- mean(model_m5_kfold$resample$RMSE)
10
11 print(c('Cross-validation RMSE' = score_m5_kfold))
```

Cross-validation RMSE
13.13364

Model 6

▼ Show/hide code

```
1  model_m6_kfold <- train(
2    formula_m6,
3    data = db_geih,
4    method = "lm",
5    metric = "RMSE",
6    trControl = ctrl_kfold
7  )
8
9  score_m6_kfold <- mean(model_m6_kfold$resample$RMSE)
10
11 print(c('Cross-validation RMSE' = score_m6_kfold))
```

Cross-validation RMSE
17.73399

The following table places the validation approach and K -fold cross-validation estimates side by side.

Model <chr>	Validation: out-of-sample <dbl>	K-fold: out-of-sample <dbl>
M0	13.97	14.06
M1	13.37	13.61
M2	12.91	13.19
M3	12.77	13.13
M4	12.75	13.13
M5	12.73	13.13

Model	Validation: out-of-sample	K-fold: out-of-sample
<chr>	<dbl>	<dbl>
M6	13.17	17.73
7 rows		

Interpretation of the K-Fold RMSE

The held-out-fold RMSE initially follows the same downward pattern as the validation RMSE. It falls from 14.06 hours in Model 0 to its minimum of 13.13 hours in Model 3. The family-composition terms in Models 4 and 5 leave predictive performance nearly unchanged. Finally, Model 6 exhibits overfitting.

The precise minimum differs from the validation-set result, which favors Model 5. The validation approach uses one split, whereas five-fold cross-validation evaluates every observation once. Consequently, the model ranking under the validation approach can be sensitive to the random seed.

LOOCV

In leave-one-out cross-validation (LOOCV), we fit each model n times, where n is the sample size. Each iteration leaves out a different observation, trains the model on the remaining $n - 1$ observations, and predicts the omitted outcome. LOOCV uses nearly all available observations for each fit and estimates the model n times, so it can be computationally intensive.

Leave-one-out cross-validation: each observation serves as the validation set once.

Leave-one-out cross-validation

Each observation forms its own validation set

	Obs. 1	Obs. 2	Obs. 3	...	Obs. n - 1	Obs. n
Run 1	Validate	Train	Train	Train	Train	Train
Run 2	Train	Validate	Train	Train	Train	Train
Run 3	Train	Train	Validate	Train	Train	Train
...				...		
Run n	Train	Train	Train	Train	Train	Validate

Repeat n times: fit on n - 1 observations and evaluate on one. Every observation is held out exactly once.

We change the resampling method in `trainControl()` and save the held-out prediction from each iteration. Unlike the validation and K -fold approaches, LOOCV uses no random partition.

▼ Show/hide code

```
1 ctrl_loocv <- trainControl(
2   method = "LOOCV",
3   savePredictions = "final"
4 )
```

Train and Evaluate the Models

For LOOCV, we compute a single RMSE from the complete vector of held-out predictions. We compare `pred` with `obs` inside the saved prediction object so that the two vectors always have the same ordering.

Model 0

▼ Show/hide code

```
1 model_m0_loocv <- train(
2   formula_m0,
3   data = db_geih,
```

```
4   method = "lm",
5   metric = "RMSE",
6   trControl = ctrl_loocv
7 )
8
9 score_m0_loocv <- RMSE(
10  pred = model_m0_loocv$pred$pred,
11  obs = model_m0_loocv$pred$obs
12 )
13 score_m0_loocv
```

[1] 14.08081

Model 1

▼ Show/hide code

```
1  model_m1_loocv <- train(
2    formula_m1,
3    data = db_geih,
4    method = "lm",
5    metric = "RMSE",
6    trControl = ctrl_loocv
7  )
8
9  score_m1_loocv <- RMSE(
10   pred = model_m1_loocv$pred$pred,
11   obs = model_m1_loocv$pred$obs
12 )
13 score_m1_loocv
```

[1] 13.63964

Model 2

▼ Show/hide code

```
1  model_m2_loocv <- train(
2    formula_m2,
3    data = db_geih,
4    method = "lm",
5    metric = "RMSE",
6    trControl = ctrl_loocv
7  )
8
9  score_m2_loocv <- RMSE(
10   pred = model_m2_loocv$pred$pred,
11   obs = model_m2_loocv$pred$obs
12 )
13 score_m2_loocv
```

```
[1] 13.16104
```

Model 3

▼ Show/hide code

```
1  model_m3_loocv <- train(  
2    formula_m3,  
3    data = db_geih,  
4    method = "lm",  
5    metric = "RMSE",  
6    trControl = ctrl_loocv  
7  )  
8  
9  score_m3_loocv <- RMSE(  
10   pred = model_m3_loocv$pred$pred,  
11   obs = model_m3_loocv$pred$obs  
12 )  
13 score_m3_loocv
```

```
[1] 13.09089
```

Model 4

▼ Show/hide code

```
1  model_m4_loocv <- train(  
2    formula_m4,  
3    data = db_geih,  
4    method = "lm",  
5    metric = "RMSE",  
6    trControl = ctrl_loocv  
7  )  
8  
9  score_m4_loocv <- RMSE(  
10   pred = model_m4_loocv$pred$pred,  
11   obs = model_m4_loocv$pred$obs  
12 )  
13 score_m4_loocv
```

```
[1] 13.08378
```

Model 5

▼ Show/hide code

```
1  model_m5_loocv <- train(  
2    formula_m5,  
3    data = db_geih,  
4    method = "lm",
```

```

5   metric = "RMSE",
6   trControl = ctrl_loocv
7 )
8
9 score_m5_loocv <- RMSE(
10  pred = model_m5_loocv$pred$pred,
11  obs = model_m5_loocv$pred$obs
12 )
13 score_m5_loocv

```

[1] 13.08449

Model 6

▼ Show/hide code

```

1  model_m6_loocv <- train(
2    formula_m6,
3    data = db_geih,
4    method = "lm",
5    metric = "RMSE",
6    trControl = ctrl_loocv
7 )
8
9 score_m6_loocv <- RMSE(
10  pred = model_m6_loocv$pred$pred,
11  obs = model_m6_loocv$pred$obs
12 )
13 score_m6_loocv

```

[1] 65.10415

This process takes longer, even though we are working with only 1342 observations. To monitor its progress, we can enable verbose output⁶.

▼ Show/hide code

```

1  ctrl_loocv_verbose <- trainControl(
2    method = "LOOCV",
3    savePredictions = "final",
4    verboseIter = TRUE
5 )
6
7 model_m6_loocv_verbose <- train(
8   formula_m6,
9   data = db_geih,
10  method = "lm",
11  metric = "RMSE",

```

```
12   trControl = ctrl_loocv_verbose
13   )
```

A Note on Leverage and LOOCV

For a regression model, we can compute the leave-one-out residual from the residual $\hat{\varepsilon}_i$ and leverage h_{ii} of the model fitted to the full sample. We therefore do not need to estimate n regressions on different samples. To see why, recall that

$$\hat{\beta} - \hat{\beta}_{(-i)} = \left(\frac{\hat{\varepsilon}_i}{1 - h_{ii}} \right) (X'X)^{-1} x_i, \text{ where } \hat{\varepsilon}_i = y_i - x_i' \hat{\beta}$$

Define the leave-one-out error as

$$\tilde{\varepsilon}_i = y_i - x_i' \hat{\beta}_{(-i)}$$

Add and subtract $x_i' \hat{\beta}$:

$$\begin{aligned} \tilde{\varepsilon}_i &= y_i - x_i' \hat{\beta} + x_i' \hat{\beta} - x_i' \hat{\beta}_{(-i)} \\ &= \hat{\varepsilon}_i + x_i' (\hat{\beta} - \hat{\beta}_{(-i)}) \end{aligned}$$

We already know the last term, so we substitute it:

$$\tilde{\varepsilon}_i = \hat{\varepsilon}_i + x_i' \left(\frac{\hat{\varepsilon}_i}{1 - h_{ii}} \right) (X'X)^{-1} x_i$$

Recall that $h_{ii} = x_i' (X'X)^{-1} x_i$. Therefore,

$$\begin{aligned} \tilde{\varepsilon}_i &= \hat{\varepsilon}_i + \frac{\hat{\varepsilon}_i h_{ii}}{1 - h_{ii}} \\ &= \frac{\hat{\varepsilon}_i}{1 - h_{ii}} \end{aligned}$$

In other words, we can recover the leave-one-out error from the full-sample OLS residual and a leverage correction. The factor $1/(1 - h_{ii})$ scales the in-sample residual because that residual is too optimistic: observation i contributed to estimating the model.

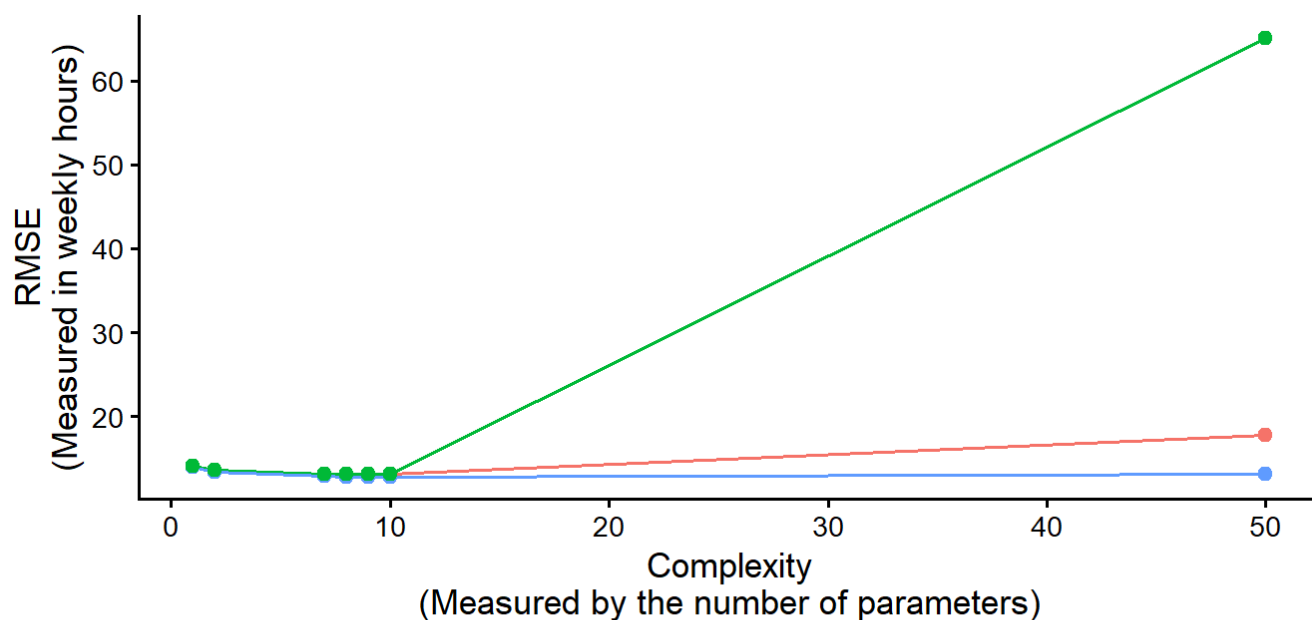
Consequently, we can calculate the exact OLS LOOCV error as

$$RMSE_{LOOCV} = \sqrt{\frac{1}{n} \sum_{i=1}^n \left(\frac{\hat{\varepsilon}_i}{1 - h_{ii}} \right)^2}$$

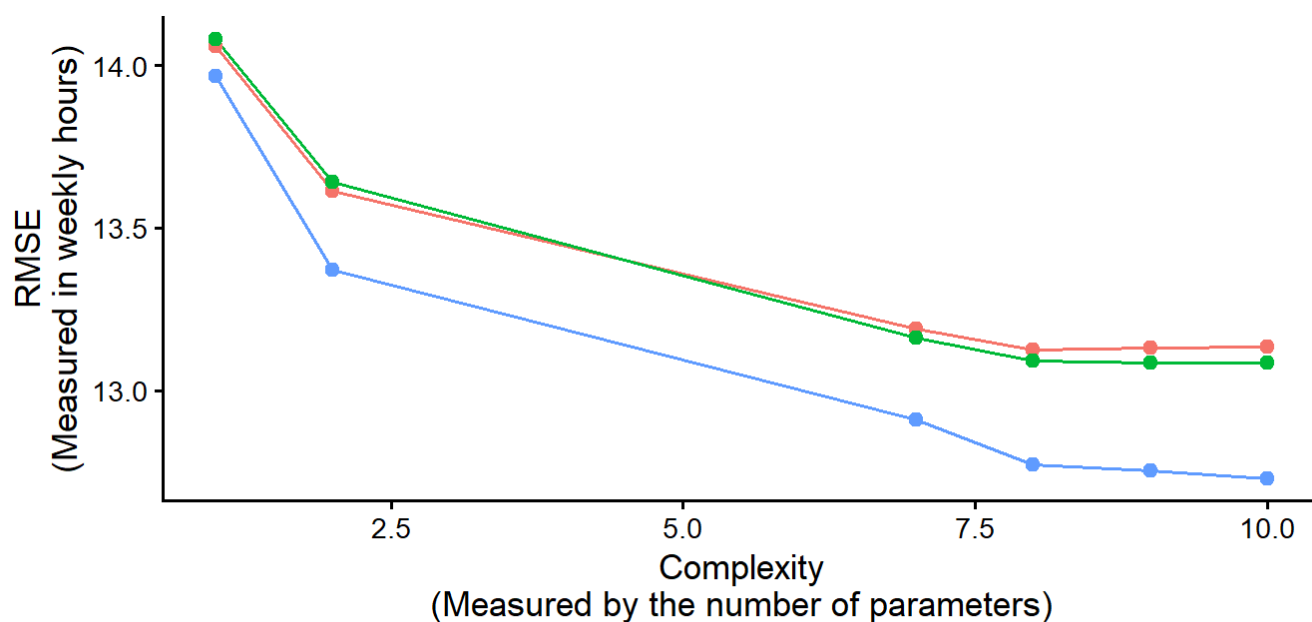
Conclusion

Model Performance Comparison

We now compare the seven models across the three approaches. Because a lower out-of-sample RMSE indicates better generalization, we look for the model that achieves the lowest value.



Assessment method — K-Fold — LOOCV — Validation



Assessment method — K-Fold — LOOCV — Validation

The out-of-sample RMSE falls after we add log income and education, while the improvements become smaller for the intermediate specifications. The validation approach favors Model 5, five-fold

cross-validation favors Model 3, and LOOCV favors Model 4. The conclusion about Model 6, however, is clear: its generalization performance deteriorates under all three assessment methods.

Additional Comments

The Effects of Changing the Data Partition

The validation estimate can change substantially when different observations are assigned to the training and validation sets. Five-fold cross-validation also uses a random partition, but it reduces this sensitivity by evaluating each observation once and averaging across folds. LOOCV uses no random partition and is therefore unchanged by the random seed.

References

- Kuhn, M. (2008). Building Predictive Models in R Using the caret Package. *Journal of Statistical Software*, 28(5), 1–26. <https://doi.org/10.18637/jss.v028.i05>
 - Kuhn, M. (2019). *The caret Package*. <https://topepo.github.io/caret/index.html>
-

Footnotes

1. The “usual” disclaimers apply. If you have comments or suggestions, please email us. They are welcome and will be considered in your final grade. [↩](#)
2. Economists typically model the intensive margin of labor supply—i.e., how many hours a given individual works—as the outcome of an optimization problem in which people trade off leisure against consumption. Given the budget constraint, consumption is financed by labor income and, potentially, non-labor income. At the optimum, the first-order condition requires that the disutility of an additional hour of work (the marginal value of forgone leisure) equals the utility derived from the additional consumption made possible by wage income. Accordingly, hours worked respond both to wages (substitution effect) and to non-labor income (income effect). For this reason, it is important to distinguish between labor and non-labor sources of income when analyzing labor supply. However, for simplicity, we will ignore that distinction for now. [↩](#)
3. The association between children and hours worked may vary with the child’s age. A richer specification could distinguish, for example, children aged 0–5 from those aged 6–17. [↩](#)
4. For instance, if we use powers of the labor income as regressors, each term can become extremely large. x is measured in 10^6 , x^2 is measured in 10^{12} , x^3 in 10^{18} , and so on. Therefore, high-order polynomials are “numerically safer” when the regressor is expressed on a moderate scale. In this case, measure the labor income in millions of pesos solves the problem. [↩](#)
5. The `sample()` function provides another way to create this partition. [↩](#)
6. We do not execute the following code because it would produce a long log and would refit Model 6 unnecessarily. [↩](#)

